

HIMANSHU SHARMA

Software Development Engineer

+91 6280317558 ♦ sharma1966himanshu@gmail.com India ♦ [linkedin.com/in/himanshu1966](https://www.linkedin.com/in/himanshu1966) ♦ github.com/s-Himansh

PROFESSIONAL SUMMARY

Software Development Engineer with 2 years of experience building Go microservices and message-based systems on Microsoft Azure and Kubernetes. Designed distributed sync engines and concurrent worker pools processing 40,000+ records daily with zero data loss, and owns backend code across 5+ production repositories. Strong in distributed systems, concurrency, observability, and cloud architecture as code.

WORK EXPERIENCE

ZopSmart Technology

Bengaluru, India

Software Development Engineer

Aug 2025 - Present

- Architected a continuous, bidirectional sync engine in Go to reconcile regulatory compliance rules with product catalogs, using bulk database operations to process 40,000+ items daily with zero data loss.
- Designed a reusable, generic partitioned worker pool that applies FNV-1a hashing to deterministically route items across goroutines, eliminating race conditions and improving parallel processing efficiency by 30%.
- Guaranteed data consistency using ACID database transactions and published near-real-time change events to an internal event streaming platform built on Azure Event Hubs, reducing downstream POS update latency by 40%.
- Authored Terraform modules to provision Azure cloud resources (Event Hubs, Blob Containers) as code, standardizing environment setup across 15+ services and reducing deployment time by 40%.
- Maintained code ownership of 5+ production repositories across multiple business domains, delivering Go microservices from design through deployment.

Software Development Engineer Intern

Aug 2024 - Jul 2025

- Developed 10+ production Go microservices from the ground up, exposing REST and gRPC APIs to support core business operations, improving system modularity and reducing deployment times.
- Owened message-based integrations by building real-time publishers on Azure Event Hubs, streaming business entity updates to 5+ downstream systems while handling 2M+ daily messages with 99.9% uptime.
- Instrumented publishers with Prometheus metrics and Grafana dashboards for real-time visibility into publish rates and latency, reducing incident response time by 25%.
- Deployed and orchestrated 20+ microservices on Kubernetes using Helm charts and ArgoCD-automated GitOps pipelines, reducing release cycle time by 50% for scalable, resilient, and repeatable releases.
- Automated secure data transmission by building Linux-based SFTP services, ensuring dependable delivery of 10K+ program-eligible item records daily to external partners with zero data loss.

EDUCATION

Chitkara University

Bachelor of Engineering (B.E.), Computer Science, 9.1/10

- CGPA: 9.1/10

2021 - 2025

Punjab, India

SKILLS & INTERESTS

Languages: Go (Golang), Java, SQL, Bash

Frameworks & APIs: Gin, GoFr, REST APIs, gRPC, Microservices

Cloud & Infrastructure: Microsoft Azure (Event Hubs, Blob Storage), AWS, Kubernetes, Docker, Helm, Terraform (Infrastructure as Code), Rancher, Linux

Databases & Messaging: MySQL, PostgreSQL, Cassandra, MongoDB, Azure Event Hubs (Kafka-compatible), Event-Driven Architecture, SFTP

DevOps & Observability: CI/CD, GitOps, GitHub Actions, ArgoCD, Harness, Prometheus, Grafana, Git, Monitoring and Alerting

Core Concepts: Distributed Systems, Concurrency (Goroutines, Channels, Mutexes), Data Partitioning and Sharding, ACID Transactions, Caching, Rate Limiting, System Design

PROJECTS

Sharded LRU Cache

- Engineered a concurrent, generic in-memory key-value cache in Go with Least Recently Used (LRU) eviction, using a hash map and doubly linked list to achieve over 50,000 requests per second.
- Reduced lock contention by 45% by partitioning the cache into 16 shards using FNV-1a hashing, with sync.RWMutex enabling concurrent reads and thread-safe evictions.

Partitioned API Throttler

- Built a high-throughput API rate limiter in Go processing 10,000+ requests per second using FNV-1a hashing to partition client requests and a sliding-window algorithm to enforce per-IP rate limits.
- Exposed custom Prometheus metrics to monitor throttle rates and latency with 99.9% visibility; containerized the service with Docker.